Tabular Q-Learning versus Deep Q-Networks: Sample Efficiency, Sparse Rewards and DQN Ablations

Aispur Santiago Amaya Bryan

Intelligent Agents — Instructor: PhD Armas Rolando

November 5, 2026

Abstract

We compare tabular Q-learning on a discretized state space with a Deep Q-Network (DQN) implemented from scratch in PyTorch. Every curve is indexed by *cumulative environment steps*, averaged over ten fixed seeds with 95% Student-t confidence bands, and evaluated greedily ($\varepsilon = 0$, frozen weights) at regular checkpoints. On CartPole-v1, DQN reaches the solved threshold in $\approx$7$\times$ fewer environment steps than the best tabular agent, but each step costs $\approx$23$\times$ more CPU, so tabular learning is $\approx$3$\times$ cheaper per unit of *compute*. On the sparse-reward MountainCar-v0, DQN with ε-greedy exploration never reaches the goal and converges to a flat $Q \approx -1/(1-\gamma)$. An optimistically initialized table solves it, while a pessimistically initialized one fails like DQN, which isolates optimism as the cause. On LunarLander-v3, ablations show how the target network and the replay buffer each contribute to stability. We close with an analysis of why neither family scales to continuous control or pixel observations.

1 Introduction

The question we study is not only *whether* an agent learns, but *why* it wins or loses under a fair experimental protocol. We hold three rules throughout. (i) Algorithms are compared on environment steps, because a tabular agent's early CartPole episodes last $\sim$20 steps while a good DQN episode lasts 500, so an "episode" axis is not a common clock. (ii) Performance is measured by periodic greedy evaluation with frozen weights, reported separately from the noisy ε-greedy training returns. (iii) Every configuration uses ten seeds, with seeding of `random`, NumPy, `torch.manual_seed` and `env.reset(seed=…)`. Negative results are reported and diagnosed rather than tuned away. All code, raw logs, weights and the command that regenerates every figure (`python experiments/run_all.py`) are in the accompanying repository.

2 Background

Tabular Q-learning. An off-policy TD control method [Sutton and Barto, 2018, Sec. 6.5]:

$$Q(s,a) \leftarrow Q(s,a) + \alpha\big[r + \gamma \max_{a'} Q(s',a') - Q(s,a)\big]. \quad (1)$$

Continuous observations must first be mapped to a finite index by binning each dimension (Sec. 3). The bootstrap term is dropped only on a *true* terminal transition. A time-limit truncation is not terminal, so we still bootstrap there.

DQN. Mnih et al. [2015] replace the table with a network $Q_\theta(s,\cdot)$ that outputs one value per action and minimise a Huber loss between $Q_\theta(s,a)$ and the target $y = r + \gamma(1-d)\max_{a'} Q_{\bar\theta}(s',a')$. Two mechanisms make this stable. An *experience replay* buffer is sampled uniformly, which breaks the temporal correlation of consecutive transitions and reuses data. A *target network* $\bar\theta$ is copied from θ only every C steps, so the regression target does not move with every gradient step. In code, $Q_\theta(s,a)$ for the actions actually taken is extracted with `q(s).gather(1, a)`; the max is used only for the next-state target.

Extensions. *Double DQN* [van Hasselt et al., 2016] decouples action selection from evaluation: $y = r + \gamma Q_{\bar\theta}(s', \arg\max_{a'} Q_\theta(s',a'))$, which reduces the overestimation bias of max. *Dueling DQN* [Wang et al., 2016] decomposes $Q(s,a) = V(s) + A(s,a) - \frac{1}{|\mathcal{A}|}\sum_{a'} A(s,a')$.

3 Experimental setup

Environments. Gymnasium 1.3 [Towers et al., 2024]: CartPole-v1 (4-D state, 2 actions, +1 per step, truncated at 500, solved at 475), MountainCar-v0 (2-D, 3 actions, -1 per step, truncated at 200) and LunarLander-v3 (8-D, 4 actions, shaped reward, solved at 200).

Discretization. Each dimension is split into k uniform bins between a lower and an upper bound. Values beyond the bounds fall into the outer bins. Position

Table 1: Hyper-parameters. Only the switch named in each configuration changes.

	CartPole / MountainCar	LunarLander
Tabular α, γ	0.1, 0.99	–
Tabular ε	$1 \to 0.01$ over 500k (CP); MC: as DQN	–
DQN net	$4/2 \to 128 \to 128$	$8 \to 256 \to 256$
Optimiser	Adam, lr $5{\cdot}10^{-4}$	Adam, lr $5{\cdot}10^{-4}$
Batch / buffer	64 / 50k	128 / 100k
Learning starts	1k	10k
Train freq.	every step	every 4 steps
Target update	hard, every 500	hard, every 1000
DQN ε	$1 \to 0.05$ over 25k (MC: 30k)	$1 \to 0.05$ over 100k
Loss / grad clip	Huber / 10	Huber / 10
Budget	CP: 1M tab., 200k DQN; MC: 500k	500k
Evaluation	10 ep. every 5k	10 ep. every 10k

Table 2: CartPole-v1 velocity magnitudes over 50k steps (random and trained policy). Both are unbounded in the observation space; our clip bounds cover every visited value. Position and angle use the termination limits.

Policy	Dimension	$\lvert x \rvert$ p99.9	$\lvert x \rvert$ max	Clip
random	cart velocity	1.943	2.829	3
random	pole ang. velocity	2.480	2.924	3.5
trained DQN	cart velocity	1.733	2.096	3
trained DQN	pole ang. velocity	1.471	1.701	3.5

(± 2.4) and pole angle ($\pm 0.2095\,\text{rad} = 12°$) use the termination limits as bounds. The velocities are unbounded in the observation space, so we *clip* them at ± 3.0 and ± 3.5. Table 2 justifies this: over 50k steps under a random policy and under the trained DQN, the largest observed values are 2.83 (cart velocity) and 2.92 (angular velocity), so no visited value exceeds the bounds and almost no bin is wasted on states that never occur. We also support hand-made boundaries per dimension, which we use to study the pole angle.

Protocol and statistics. Hyper-parameters are in Table 1. At each checkpoint we run 10 greedy episodes on a separate environment, from the same evaluation seeds at every checkpoint, so that changes in the curve reflect the policy rather than the sampled start states. We report the mean over 10 seeds with a 95% Student-t interval ($t_{9,0.975} = 2.26$, deliberately wider than the normal 1.96). Plotted greedy curves use a short trailing average over checkpoints (5 for CartPole, 3 elsewhere), applied per seed before aggregation; the tables use raw values. Summary metrics, computed per seed: *final* (mean of the last 3 checkpoints), *best*, *AUC* (mean over all checkpoints, which rewards speed and stability), *drop* = best − final (forgetting) and steps to threshold. Every run uses one CPU thread. Seeds 0–4 (and 8–9 for LunarLander) ran on a laptop (Ryzen 5 7235HS) and the others on 4-vCPU GitHub Codespaces with identical pinned packages. Learning metrics use all seeds, and CPU-time figures use only the laptop runs so that timings come from one CPU.

4 Tier 1: tabular Q-learning vs. DQN

4.1 CartPole-v1

Sample efficiency. Fig. 1 and Table 3 show the core trade-off. With the best decay schedule, DQN first reaches a greedy return $\geq$475 after 56 ± 16k environment steps (10/10 seeds). The fastest table (10 bins) needs 382 ± 58k, so DQN is $\approx$6.9$\times$ more sample-efficient. At an *equal* budget of 200k steps, DQN (ε decay 5k) scores 356 ± 95 against 175–215 for the three tables (Welch $t = 4.2$, $p = 0.002$ vs. 10 bins). The mechanism is generalization: a gradient step on one state also moves Q at nearby states, whereas a table cell learns only from its own visits.

Compute. The comparison reverses on CPU time. Measured on the same laptop, a tabular step costs 0.097 ms and a DQN step 2.24 ms (a forward pass to act plus a backward pass on a 64-sample batch), a factor of 23. Reaching 475 therefore costs $\approx$37 CPU-seconds for the table and $\approx$124 for DQN: tabular learning is 3.4$\times$ cheaper in *compute*. DQN is sample-efficient but compute-hungry, and tabular Q-learning is the reverse. Which one is "faster" depends on whether environment interaction or computation is the scarce resource.

Number of bins. More bins do not monotonically help. Six bins (1,296 states) visit 39% of the table, ten bins (10,000) 18%, and twenty bins (160,000) only 4.6%. Ten bins is the best compromise. It is the fastest to the threshold (382 ± 58k steps, 10/10 seeds) and has the highest final score (392 ± 81). Twenty bins is the worst on every metric: final 263 ± 96, only 7/10 seeds ever reach 475, the largest drop after the peak (212 ± 88) and the largest seed-to-seed spread. Six bins learns reliably (10/10) but more slowly (518 ± 59k), because it cannot represent the policy as precisely. The difference between 6 and 20 bins in final score is suggestive but not significant at 5% ($p = 0.07$), so we state it as a trend. The mechanism is the classic bias–variance trade-off of state aggregation: coarse cells generalize "for free" but cap precision, and fine cells are precise but data-starved. After 1M steps ($\approx$10k episodes) 95% of the 20-bin table has never been updated.

Pole-angle boundaries. Fig. 2 fixes the other dimensions at 10 bins and changes only the angle bins. With two angle bins (only the *sign* of the tilt) the agent cannot tell a slight tilt from a dangerous one. Over the first 200k steps it averages 107 ± 8 against 165 ± 14 for 10 uniform bins (Welch $p < 10^{-6}$), and its AUC is the lowest (226 vs. 294). Putting 10 bins inside $|\theta| < 0.1$ rad, where the pole spends nearly all its time, learns more slowly at first (120 ± 10) because the fine cells need more visits,

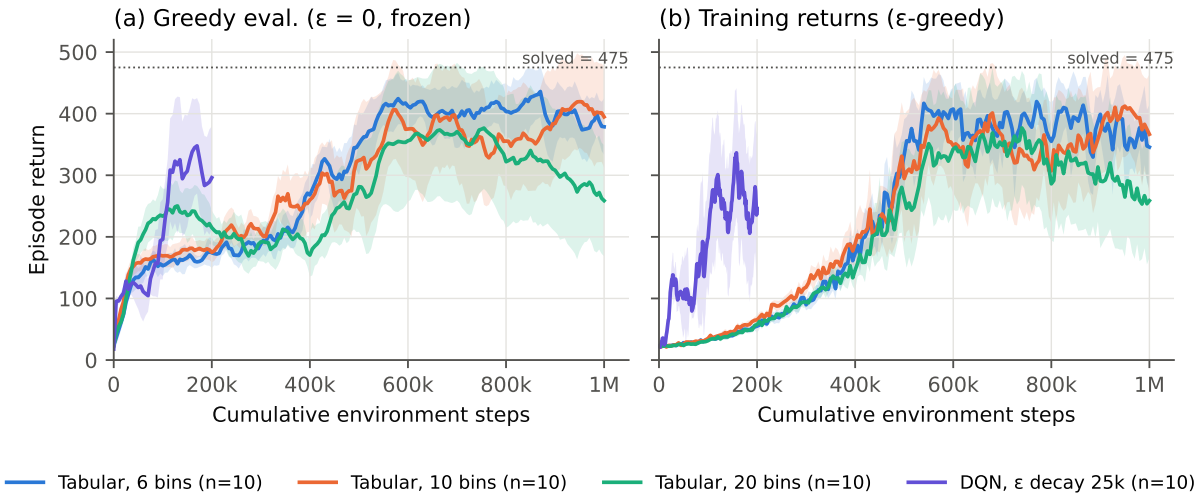


Figure 1: CartPole-v1 on a common x-axis of environment steps. (a) Greedy evaluation; (b) training returns (20-episode moving average). DQN (200k budget) rises within the first ∼100k steps, while the tables need ∼400–550k. Bands: 95% CI over 10 seeds; greedy curves smoothed over 5 checkpoints.

Table 3: Tier 1, CartPole-v1. Greedy-evaluation returns (10 episodes per checkpoint, every 5k steps), mean ± 95% t-CI over seeds. Final = mean of the last 3 checkpoints; AUC = mean over all checkpoints; Drop = best − final (forgetting); Early = mean greedy return over the first 50k steps. †CPU time from the runs on the reference laptop only (`results/hosts.json`).

Config	Steps	Final	Best	AUC	Early (≤50k)	Drop	k-steps to 475 (solved)	States seen	CPU s/run†
Tabular, 6 bins	1000k	355.8 ± 51.0	494.1 ± 6.4	302.7 ± 10.5	113.9	138.2 ± 49.1	518 ± 59 (10/10)	38.7%	101
Tabular, 10 bins	1000k	392.2 ± 81.4	499.8 ± 0.4	293.8 ± 29.6	140.2	107.6 ± 81.4	382 ± 58 (10/10)	18.0%	97
Tabular, 20 bins	1000k	263.0 ± 95.8	474.9 ± 31.3	268.5 ± 46.5	145.7	211.9 ± 88.1	505 ± 70 (7/10)	4.6%	94
Angle: 2 bins (sign only)	1000k	305.1 ± 73.2	486.0 ± 23.9	226.2 ± 23.4	92.7	180.9 ± 77.8	608 ± 159 (8/10)	22.9%	88
Angle: fine near 0	1000k	362.4 ± 73.5	476.7 ± 25.0	274.3 ± 30.5	51.6	114.3 ± 61.7	474 ± 114 (6/10)	16.0%	86
DQN, ε decay 5k	200k	340.2 ± 134.1	500.0 ± 0.0	324.3 ± 36.4	189.8	159.8 ± 134.1	56 ± 16 (10/10)	–	449
DQN, ε decay 25k	200k	296.6 ± 114.7	500.0 ± 0.0	223.2 ± 33.6	123.0	203.4 ± 114.7	98 ± 21 (10/10)	–	427
DQN, ε decay 100k	200k	139.8 ± 65.9	309.4 ± 91.4	102.7 ± 9.4	122.0	169.6 ± 61.8	180 ± 191 (2/10)	–	391

and ends level with uniform binning (final 362 vs. 392, CIs overlap). Resolution only pays off where the agent has enough data to fill it.

ε-decay and forgetting. Fig. 3 varies only the length of the linear ε decay. On CartPole, fast decay (5k steps) is best both early (mean greedy return 190 ± 52 over the first 50k steps) and late (379 ± 48 after 150k), and 10/10 seeds reach 475. CartPole needs little exploration: random actions already visit the relevant near-upright states, so the sooner the agent exploits, the sooner its data comes from good trajectories. Slow decay (100k) spends half the budget acting mostly at random. Its training returns stay low until ε is small, and only 2/10 seeds reach 475 within 200k steps. Every fast- and medium-decay seed (20/20) also shows *catastrophic forgetting*: the greedy return reaches ≥475 and later falls below 200, with mean drops of 160–200. Once the policy is good, the 50k-transition buffer fills with near-upright states only, and the network overwrites what it knew about recovering from large tilts. This is why performance must be tracked *during* training rather than only at the end: a final-checkpoint evaluation alone would report 340 ± 134 for a method that had already solved the task.

4.2 MountainCar-v0: a planned negative result

MountainCar pays −1 on every step, and an episode ends at the flag or after 200 steps. To isolate the cause of failure, all four agents share the same ε schedule ($1 \to 0.05$ over 30k steps, or 150k for the second DQN). Fig. 4 and Table 4 summarize the result.

Optimism, not tabular-ness, is what explores. With $Q_0 = 0$, every true value (< 0) lies below the initial estimate. Each visit lowers the tried action's value, so untried actions look better, and the agent sweeps the state space systematically. It reached the flag on *10/10* seeds (median first success at 113k steps) and its greedy return improved to -176 ± 10 (best -148 ± 6). The *same* agent with $Q_0 = -100 = -1/(1-\gamma)$ has no optimism bonus and failed on *all* 10 seeds, with a greedy return of exactly −200 throughout. The decisive variable is the initialization, not the function class.

Table 4: MountainCar-v0 (sparse reward, 500k steps, same ε schedule for all). Greedy returns are mean ± 95% CI; -200 means the flag was never reached.

Config	Final greedy	Best greedy	Seeds reaching flag	Median first flag	Successful train eps	CPU s/run
Tabular, Q0 = 0 (optimistic)	-176.2 ± 9.6	-147.6 ± 6.0	10/10	113.1k	703	39
Tabular, Q0 = -100	-200.0 ± 0.0	-200.0 ± 0.0	0/10	–	0	46
DQN, ε decay 30k	-188.7 ± 17.4	-168.3 ± 34.0	3/10	93.8k	595	1038
DQN, ε decay 150k	-174.2 ± 30.2	-163.8 ± 31.5	3/10	118.0k	514	793

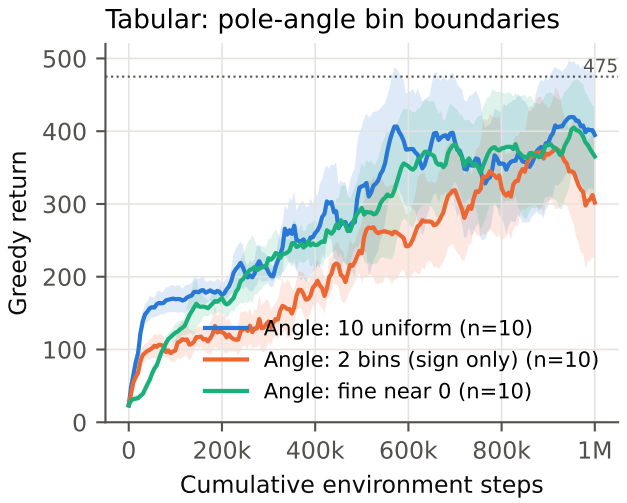


Figure 2: Tabular CartPole: only the pole-angle boundaries change; all other dimensions use 10 bins.

Why DQN usually fails. Until the first success, every transition in the buffer has $r = -1$, so every target is $-1 + \gamma \max Q$, which converges to $-1/(1-\gamma) = -100$ in every state. In the 14/20 DQN runs that never found the flag, the final mean Q is -99.4 ± 0.1 and the greedy return is -200. The network has learned that "all states are equally bad": the value landscape is flat, so no gradient points toward the goal. Exploration is left to ε-greedy dithering, which almost never produces the ∼100 coordinated back-and-forth pushes needed to build momentum. A longer ε decay did not help (3/10 vs. 3/10 seeds reach the flag), because more *undirected* noise is not more exploration.

What happens when DQN does see the flag. In 6/20 runs a lucky random trajectory reached the flag. Those runs then learned better greedy policies than the table (-138 ± 38), but their value estimates *diverged*. In MountainCar every true Q is ≤ 0, yet after the first success four of the six runs climbed above +4,900 (up to $9.6 \cdot 10^4$), one reached +79 and only one stayed below zero. The pre-clip gradient norm rose from ≈0.4 to as much as $7.9 \cdot 10^3$. This is the "deadly triad" of function approximation, bootstrapping and off-policy updates [Sutton and Barto, 2018, Sec. 11.3]. The flag creates a sharp value cliff between -1 and -100 that the MLP cannot represent locally. The error spreads to nearby states and is then amplified by the max in every bootstrap target. The policy can stay reasonable because arg max depends only on how the values rank, not on their scale. A tabular update can never diverge like this, because each update moves only its own cell toward its target. A unit test (`test_dqn_targets_terminal_and_bootstrap`) confirms that terminal transitions use $y = r$ exactly, which rules out a terminal-handling bug.

5 Tier 2: DQN ablations on LunarLander-v3

The full agent solves LunarLander on 10/10 seeds, with a final greedy return of 234 ± 29 (Table 5, Fig. 5). We then remove one stabilizer at a time and keep every other hyper-parameter fixed.

No target network: learns, but cannot stay good. Bootstrapping from the online network makes the regression target move with every gradient step. Early learning is just as fast. The peak greedy return matches the full agent (275 ± 7 vs. 273 ± 6), and every seed reaches 200 at some point. But the agent cannot hold its policy. All 10/10 seeds that reached ≥200 later fell below 0, against 2/10 for full DQN. The drop from peak to final is 99 ± 35 (full: 39 ± 29), and the final score is lower (176 ± 37, Welch $p = 0.014$). The TD loss stays higher throughout (Fig. 6a): the network keeps chasing a target that its own updates move. In this setting the target network buys *stability* rather than speed.

No replay buffer: never really learns. Online DQN takes one gradient step per environment step on the transition that just happened. Consecutive LunarLander states are almost identical, so the mini-batches are strongly correlated and far from i.i.d. Each transition is also used once and discarded. The agent improves briefly to a best of 149 ± 28, then degrades to a final -376 ± 188 (Welch $p < 10^{-4}$ vs. full), and only 2/10 seeds ever touch 200. Its diagnostics show the failure. The mean batch Q inflates to 257, 5.9× the full agent's, while its policy is much worse, which is clear overestimation. Its gradient norms reach ∼300 before clipping, against ∼20 for full DQN, and its loss never comes down

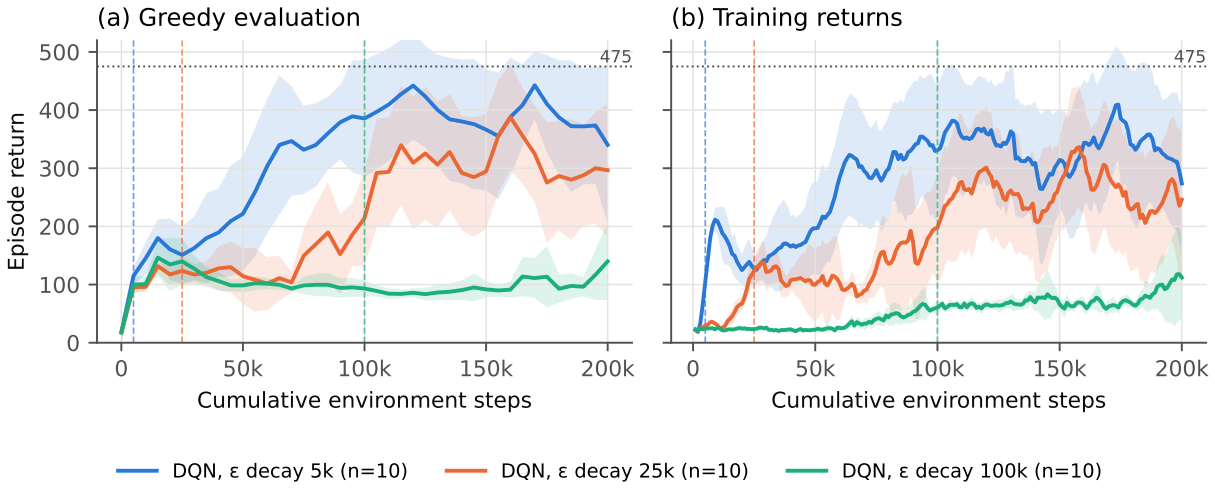


Figure 3: DQN on CartPole with three linear ε-decay lengths (dashed lines mark the end of each decay).

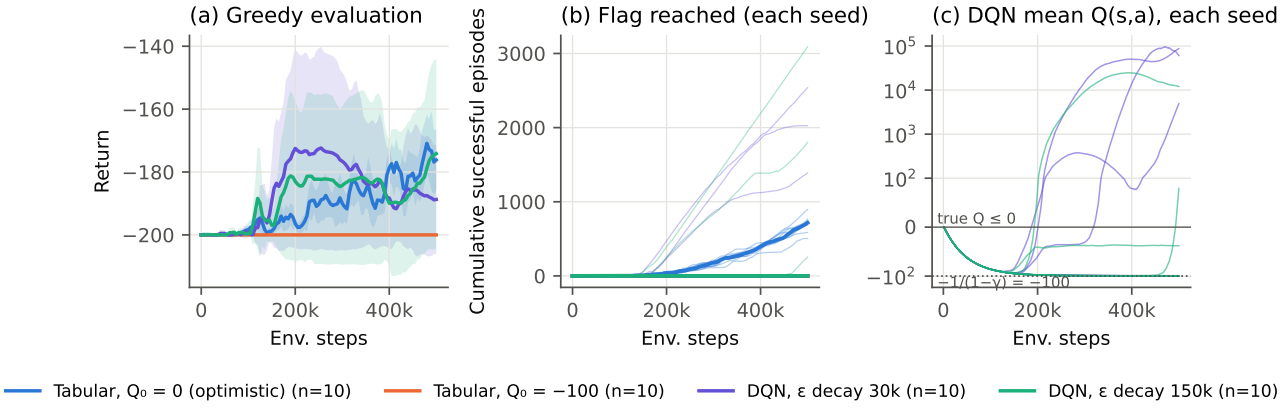


Figure 4: MountainCar-v0, 500k steps, identical ε schedule for all agents. (a) Greedy return (mean $\pm$ 95% CI). (b) Cumulative flag-reaching training episodes; thin lines are seeds, the thick line is the median. (c) Mean $Q(s,a)$ on DQN training batches per seed (symmetric-log axis). Before the flag every run sits at $-1/(1-\gamma) = -100$, and after it most runs diverge to impossible positive values.

(Fig. 6). Each update fits the current stretch of trajectory and overwrites the rest (catastrophic interference). Its CPU cost is also the highest (24 vs. 10 min per run), because it updates on every step instead of every fourth.

Double and Dueling DQN (Tier 3B). Neither extension changed the outcome measurably at this scale. Final returns were 234 ± 29 (Double) and 238 ± 33 (Dueling) against 234 ± 29 for full DQN ($p = 0.98$ and 0.82), and all three solved 10/10 seeds. Double DQN also left the Q level unchanged (43.2 vs. 43.4). With 4 actions, a target network refreshed every 1,000 steps, and a Huber loss, the overestimation that Double DQN removes is already small. Its benefit reported by van Hasselt et al. [2016] was on Atari, with 18 actions and much noisier value estimates. We report this as a null result rather than tuning until a difference appears.

Hyper-parameter study. We varied two parameters one at a time around the default, with three values each and 5 seeds (seeds 0–4, the same seeds for every row; Fig. 7, Table 6). Only a too-small learning rate hurts significantly. With lr $= 10^{-4}$ the agent learns too slowly for the 500k budget: its final score is 106 ± 117 against 258 ± 17 (Welch $p = 0.022$), and only 3/5 seeds ever reach 200. A larger rate ($2 \cdot 10^{-3}$) reaches 200 sooner (118k vs. 200k steps) but ends lower and more variable (234 ± 48, $p = 0.26$). The target-update period behaves the same way. Copying every 250 steps learns faster (146k) and forgets more (drop 53 vs. 15). Copying every 4,000 steps learns slower (252k) but is steadier than 250. Neither final score differs significantly from the default ($p = 0.22$ and 0.27). Both parameters therefore trade learning speed against stability, and removing the target network (Sec. 5) is the extreme case of a period of one step.

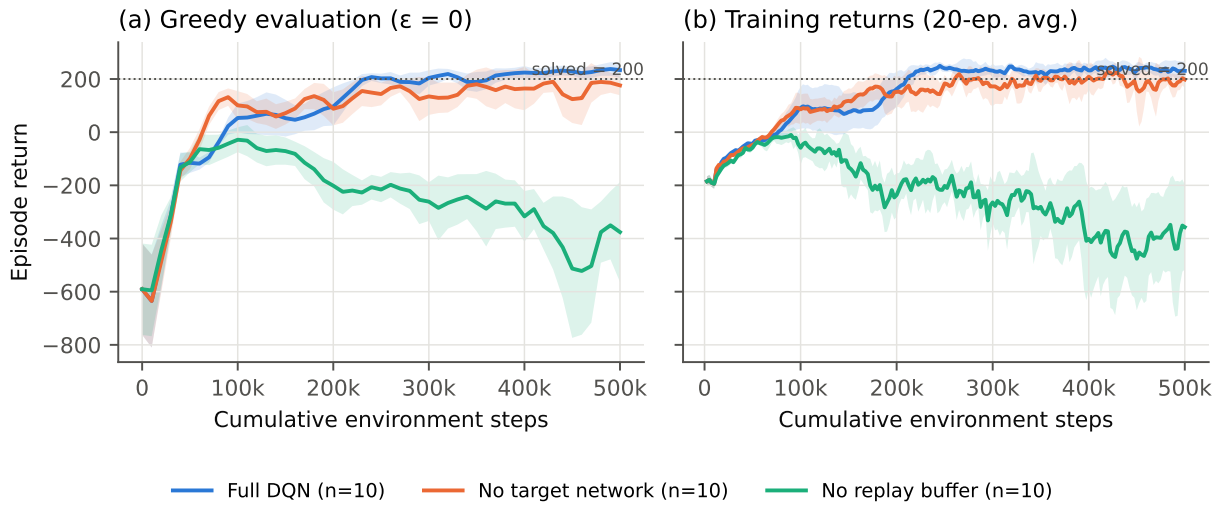


Figure 5: LunarLander-v3 ablations, 500k steps, 10 seeds. Identical hyper-parameters except the removed component.

Table 5: Tier 2, LunarLander-v3, 500k steps, greedy evaluation every 10k steps (10 episodes). Mean ± 95% t-CI over seeds. Final mean Q = average Q(s,a) on training batches over the last 50k steps.

Config	Final	Best	AUC	Drop	Mean steps to 200 (seeds)	Final mean Q	CPU min/run
Full DQN	233.7 ± 29.3	272.9 ± 5.5	104.1 ± 18.5	39.3 ± 29.4	170k (10/10)	43.4	10.4
No target network	176.1 ± 37.2	274.8 ± 7.3	88.0 ± 21.6	98.7 ± 35.1	108k (10/10)	48.4	10.0
No replay buffer	-375.9 ± 188.1	149.2 ± 27.8	-241.8 ± 43.8	525.1 ± 190.6	80k (2/10)	256.9	24.0
Double DQN	234.2 ± 28.8	276.1 ± 5.2	95.4 ± 15.5	41.9 ± 28.8	191k (10/10)	43.2	10.2
Dueling DQN	238.2 ± 33.2	275.9 ± 6.3	93.8 ± 25.2	37.7 ± 28.6	189k (10/10)	46.9	9.8

Official 100-episode evaluation. The thresholds are defined over 100 episodes, whereas our curves use 10 episodes per checkpoint and count a seed as solved when *any* checkpoint reaches the threshold. We therefore reloaded the *final* weights of every run and played 100 greedy episodes from held-out start states (`experiments/eval100.py`, Table 7). Full DQN on LunarLander passes (mean 227; 7/10 seeds $\geq$ 200). On CartPole, only 4/10 tabular (10 bins) and 5/10 DQN (ε decay 5k) final policies average $\geq$ 475. Every seed had reached 475 at some checkpoint, so this is the forgetting of Sec. 4.1 measured on the final weights. On MountainCar, the optimistic table reaches the flag in 64% of the evaluation episodes (DQN: 19%), taking ≈160 steps when it does, so it falls short of the −110 threshold (0/10 seeds). We therefore claim that it *finds the goal*, not that it solves the task.

6 Compute cost

The full study comprises 170 runs and 22.3 CPU-hours (one thread per run). On the reference laptop, per run, a 1M-step CartPole table takes ≈100 s, a 200k-step CartPole DQN ≈450 s, and a 500k-step LunarLander DQN ≈10 minutes (24 without replay). The deep agents account for most of the budget even though they use 5× fewer environment steps on CartPole. This is the sample-efficiency vs. compute trade-off of Sec. 4.1, made concrete.

7 Analysis: limits of tabular RL and DQN in complex environments

Discretizing continuous actions. DQN needs a finite action set, because its output layer has one node per action and acting requires $\max_a$. HalfCheetah has a 6-D continuous action in $[-1, 1]^6$. With only 5 levels per joint, the joint action set has $5^6 = 15{,}625$ elements. The output layer grows accordingly, $\max_a$ must scan all of them, and, most importantly, each $Q(s, a)$ is learned from the few samples in which that *exact* combination was tried. ε-greedy exploration over 15,625 arms is hopeless, and the coarse grid still cannot express smooth torques. Ant (8-D) would need $5^8 \approx 3.9 \cdot 10^5$ actions. The action space itself suffers the curse of dimensionality.

Discretizing continuous states. The table needs k^d cells. HalfCheetah's 17-D observation with $k = 10$ gives 10^{17} cells per action. That is roughly $8 \cdot 10^{17}$ bytes per action for `float64`, far beyond any memory. Even if memory were free, filling each cell even once would take

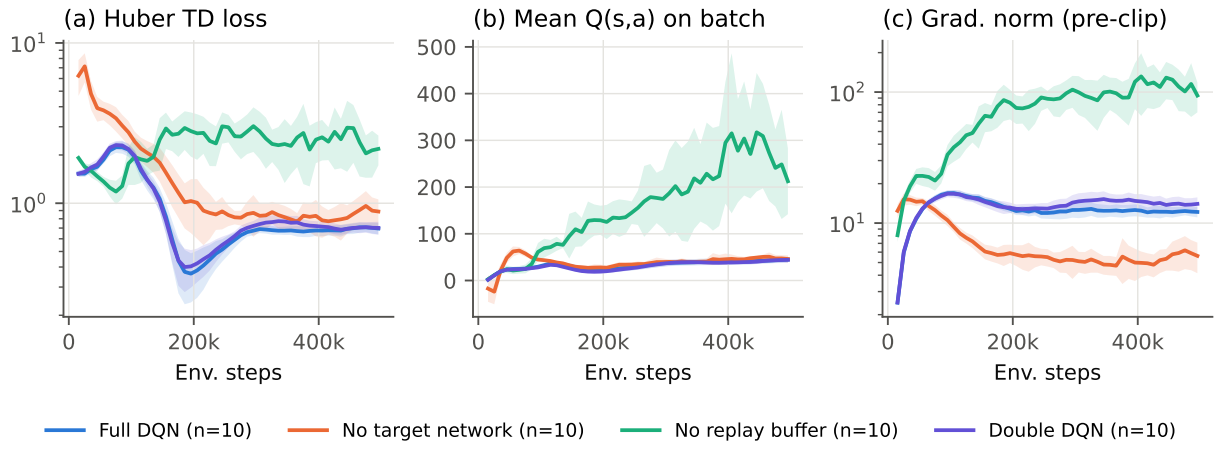


Figure 6: Training diagnostics (10k-step block averages): Huber TD loss (log), mean $Q(s, a)$ on training batches, and pre-clip gradient norm (log).

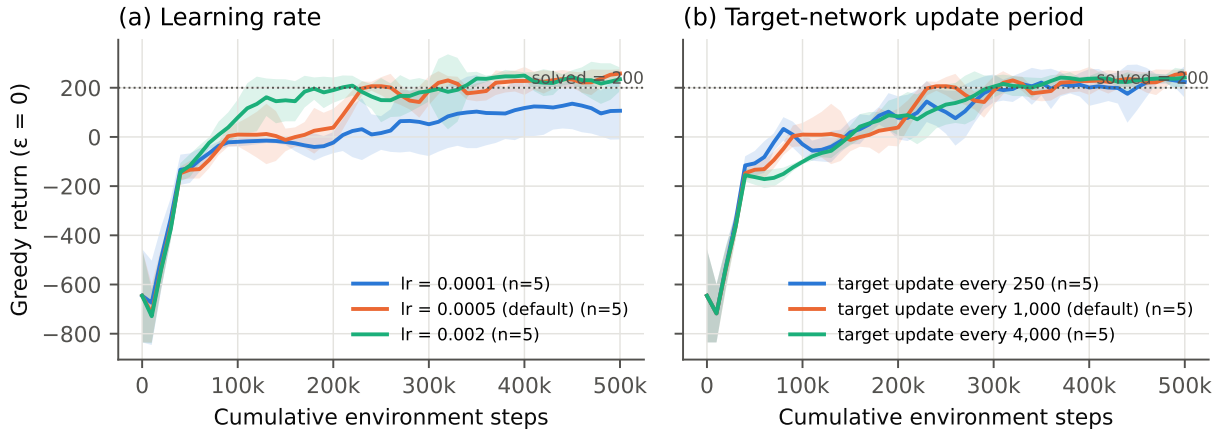


Figure 7: LunarLander-v3 hyper-parameter study (greedy evaluation, 5 seeds, 95% CI). One parameter is varied at a time, and all other settings are as in Full DQN.

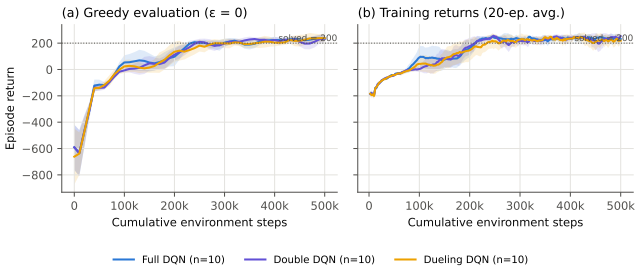


Figure 8: Double and Dueling DQN vs. the full agent: indistinguishable within the 95% CIs.

more interactions than could ever be collected. Our CartPole results already show the trend in 4-D: going from 6 to 20 bins multiplied the table by 123, and the fraction of states ever visited fell from 39% to 4.6%. Coarse coding and tile coding [Sutton and Barto, 2018, Ch. 9] soften this by letting states share features, but the number of tiles still grows exponentially with d.

Algorithms for continuous control. The remedy is to parameterize the *policy* directly and use a learned critic (actor–critic methods), which avoids $\max_a$ over an enumerated set. **PPO** [Schulman et al., 2017] is on-policy. Its actor outputs the mean (and log-std) of a Gaussian over actions, and its clipped surrogate objective limits the size of each policy update. **SAC** [Haarnoja et al., 2018] is off-policy and stochastic. It maximizes return plus policy entropy, which gives strong, automatically tuned exploration and good sample efficiency on MuJoCo. **TD3** [Fujimoto et al., 2018] is off-policy and deterministic. Its actor outputs the action vector, and twin critics with a min, delayed actor updates and target-policy smoothing counter the same max-overestimation bias that Double DQN addresses in the discrete case.

Why tables fail on pixels. An Atari frame is 210×160 RGB. Even downsampled to 84×84 grayscale with

Table 6: Tier 2 hyper-parameter study, LunarLander-v3, 500k steps, one parameter varied at a time around the default (all other settings as Full DQN). Greedy evaluation every 10k steps (10 episodes). Mean ± 95% t-CI over seeds.

Parameter	Value	Final	Best	AUC	Drop	Mean steps to 200 (seeds)
lr	0.0001	106.4 ± 116.6	189.9 ± 131.9	7.5 ± 56.9	83.5 ± 85.5	293k (3/5)
lr	0.0005 (default)	257.6 ± 17.4	272.3 ± 9.7	86.2 ± 21.6	14.7 ± 12.4	200k (5/5)
lr	0.002	234.4 ± 48.2	281.1 ± 6.3	123.2 ± 27.9	46.8 ± 49.6	118k (5/5)
target_update_freq	250	223.5 ± 64.4	276.8 ± 8.2	77.7 ± 18.6	53.3 ± 62.4	146k (5/5)
target_update_freq	1000 (default)	257.6 ± 17.4	272.3 ± 9.7	86.2 ± 21.6	14.7 ± 12.4	200k (5/5)
target_update_freq	4000	241.0 ± 33.4	274.4 ± 4.7	71.6 ± 17.4	33.4 ± 31.9	252k (5/5)

Table 7: Final weights, 100 greedy episodes per seed on held-out start states. Mean ± 95% t-CI over seeds of each seed's 100-episode mean; a seed passes when its own 100-episode mean reaches the official threshold. Main configurations only; every run is in results/tables/t_final100.csv.

Env	Config	100-ep mean (final weights)	Best seed	Seeds ≥ threshold	Threshold
CartPole-v1	dqn_eps5k	300.2 ± 153.5	500.0	5/10	475
CartPole-v1	tabular_bins10	384.6 ± 87.5	496.1	4/10	475
MountainCar-v0	dqn	-188.4 ± 17.9	-131.6	0/10	-110
MountainCar-v0	tabular_optimistic	-174.2 ± 11.4	-151.4	0/10	-110
LunarLander-v3	full	226.9 ± 34.3	272.9	7/10	200
LunarLander-v3	no_replay	-442.2 ± 303.1	-125.9	0/10	200
LunarLander-v3	no_target	188.2 ± 69.6	281.4	6/10	200

256 intensities, the number of distinct observations is $256^{7056} \approx 10^{16,992}$. A table learns only about states it has visited *exactly*, and the chance of seeing the same frame twice (outside a few reset states) is essentially zero, so no update would ever be reused. What makes Atari tractable is *generalization*: a convolutional network maps visually similar frames to similar features, so experience transfers between states that are never identical. That is the same mechanism that gave DQN its ≈7× sample-efficiency advantage on CartPole, applied at a far larger scale [Mnih et al., 2015].

8 Conclusion

Tabular Q-learning and DQN fail and succeed for identifiable reasons. On CartPole, DQN's ability to generalize between states made it ≈7× more sample-efficient, but it cost 23× more CPU per step, so the table was cheaper in compute. DQN also forgot solved policies, which only shows up when evaluation is tracked *during* training. The table's performance depends on a precision/data trade-off in the discretization. MountainCar showed that what explores is optimism, not the function class. A pessimistic table fails exactly like DQN. DQN fails because a reward that is −1 everywhere flattens its value function, and when it does reach the flag, its estimates can diverge (the deadly triad). On LunarLander, the target network was needed to *keep* a good policy and the replay buffer to *find* one at all. Double and Dueling heads gave no measurable gain at this scale. Finally, none of these methods transfers to continuous control or pixel observations without replacing enumeration by generalization (Sec. 7).

AI-use statement

We used Claude (Anthropic), through the Claude Code assistant, for code generation, debugging, and writing support. This included drafting experiment scripts and helpers, checking the repository against the project requirements, and editing this report. Every number in the text, tables and figures is produced by our own scripts (`analysis.py`, `experiments/*.py`) from the logged runs in `results/`. No result was taken from the assistant without being regenerated by that code. Both authors reviewed the AI-generated code, and we take responsibility for all of it. The most important prompts are listed in Appendix A.

References

Scott Fujimoto, Herke van Hoof, and David Meger. Addressing function approximation error in actor-critic methods. In *International Conference on Machine Learning (ICML)*, 2018.

Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *International Conference on Machine Learning (ICML)*, 2018.

Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare,

Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *Nature*, 518:529–533, 2015.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. In *arXiv preprint arXiv:1707.06347*, 2017.

Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2nd edition, 2018.

Mark Towers, Ariel Kwiatkowski, Jordan Terry, John U. Balis, Gianluca De Cola, Tristan Deleu, Manuel Goulão, Andreas Kallinteris, Markus Krimmel, Arjun KG, et al. Gymnasium: A standard interface for reinforcement learning environments, 2024. arXiv:2407.17032.

Hado van Hasselt, Arthur Guez, and David Silver. Deep reinforcement learning with double Q-learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2016.

Ziyu Wang, Tom Schaul, Matteo Hessel, Hado van Hasselt, Marc Lanctot, and Nando de Freitas. Dueling network architectures for deep reinforcement learning. In *International Conference on Machine Learning (ICML)*, 2016.

A Most important AI prompts

1. *Requirements check.* We pasted the full project statement and the repository link and asked: "Is everything here? We are a group of 2." The assistant listed the missing items (Tier 2 hyper-parameter study, agent videos, AI-use statement, 100-episode evaluation), and we asked it to implement them.
2. *Tier 2 hyper-parameter study.* Asked to add a one-at-a-time study of the learning rate and the target-update period around the default LunarLander configuration, reusing the existing runner, seeds and table format (`experiments/tier2_hparams.py`).
3. *Final evaluation and videos.* Asked for a script that reloads the final weights of every run and plays 100 greedy episodes against the official thresholds (`experiments/eval100.py`), and for GIFs of a representative (median) seed per agent (`experiments/make_gifs.py`).